Building a Question Answering System from Scratch Using the Stanford SQuAD Dataset

1 Problem Statement

Question Answering (QA) is a Natural Language Processing task where a system reads a paragraph and returns the exact answer to a given question.

Example: Paragraph: "Hyderabad was founded in 1591 by Muhammad Quli Qutb Shah. It is the capital of Telangana."

Question: "When was Hyderabad founded?"

Answer: 1591

This is called Machine Reading Comprehension. It is harder than it looks — the question and the paragraph rarely use the same words, so the model must understand the meaning behind both and connect them.

In this project we build a transformer-based QA system using PyTorch. The model takes a question and a paragraph as input and identifies the exact span of text that answers the question.

We train the system in three stages. First, the model is pretrained on Wikipedia text to learn general English language patterns. Second, it is fine-tuned on the SQuAD 1.1 dataset, which contains 87,599 question-answer pairs from Wikipedia, to learn extractive question answering. As an extension, a decoder is added so the model can generate a complete sentence answer instead of extracting a word directly from the paragraph. The system is evaluated using Exact Match and F1 score on the SQuAD validation set.

2 System Architecture

The architecture of the proposed system consists of the following components:

Input Layer

The input consists of a question and a context paragraph combined into a single token sequence.

Embedding Layer

This layer converts tokens into dense vector representations using token embeddings, positional embeddings, and segment embeddings.

Transformer Encoder

The core of the model consists of stacked transformer encoder layers. Each layer includes:

- Multi-head self-attention mechanism
- Feedforward neural network
- Layer normalization
- Residual connections

Pretraining Heads

During pretraining, two prediction heads are used:

- Masked Language Modeling head
- Next Sentence Prediction head

Question Answering Head

During fine-tuning, a linear layer predicts the start and end token positions of the answer in the context paragraph.

3 Phase-wise Implementation Plan

The project will be implemented in three major phases.

3.1 Phase 1: Language Model Pretraining

In this phase, the transformer encoder learns general language representations using self-supervised tasks.

Masked Language Modeling (MLM)

A portion of the input tokens (typically 15%) are randomly masked. The model learns to predict the original words based on surrounding context.

> **Example**
>
> *Input*
> Hyderabad was founded in [MASK] by Muhammad Quli Qutb Shah.
>
> *Target*
> 1591

This task enables the model to understand contextual relationships between words.

Next Sentence Prediction (NSP)

The model receives two sentences and predicts whether the second sentence logically follows the first in the original text.

> **Example**
>
> *Sentence A*
> Hyderabad was founded in 1591.
>
> *Sentence B*
> Muhammad Quli Qutb Shah established the city.
>
> *Output*
> True

This task helps the model understand relationships between sentences.
The encoder trained in this phase becomes the base language model.

3.2 Phase 2: Question Answering Fine-tuning

In this phase, the pretrained transformer encoder is adapted for the extractive question answering task.
The input format is structured as follows:

```
[CLS] Question [SEP] Context Paragraph [SEP]
```

The model processes the combined sequence and produces two probability distributions:

- Start position of the answer

- End position of the answer

The predicted answer span is extracted from the context paragraph using these indices.

Example

Question
When was Hyderabad founded?

Context
Hyderabad was founded in 1591 by Muhammad Quli Qutb Shah.

Prediction
Start = 5
End = 9

Answer
1591

The training objective minimizes the cross entropy loss between predicted and true start/end positions.

3.3 Phase 3: Optional Answer Generation (Advanced)

As an extension, a transformer decoder can be added to generate natural language answers instead of predicting spans.

In this case, the encoder processes the question and context, and the decoder generates the final answer sentence.

Example output
Hyderabad was founded in 1591.

This phase converts the system from an extractive QA model into a generative QA system.

4 Datasets

Two categories of datasets will be used in this project.

4.1 Pretraining Dataset

A large unlabeled text corpus is required for training the language model.

Possible sources include:

- Wikipedia text dump
- Natural Questions (google)

These datasets provide large volumes of natural language text for self-supervised learning.

4.2 Fine-tuning Dataset

The Stanford Question Answering Dataset (SQuAD) will be used for training and evaluating the QA model.

SQuAD consists of thousands of Wikipedia articles with human-generated questions and corresponding answers.

Each data sample contains:

- Context paragraph
- Question
- Answer text
- Answer start index

Example

Context
Stanford University was founded in 1885.

Question
When was Stanford University founded?

Answer
1885

5 Evaluation Metrics

The performance of the QA model will be evaluated using standard SQuAD metrics.

Exact Match (EM)

This metric measures the percentage of predictions that exactly match the ground truth answer.

F1 Score

This metric measures the overlap between the predicted answer tokens and the ground truth answer tokens.

It considers both precision and recall.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These metrics provide a reliable evaluation of model performance for extractive question answering tasks.